

8259A Programmable Interrupt Controller

What These Notes Cover

These notes explain how the **8259A PIC** is programmed using:

- **Initialization Command Words (ICW1–ICW4)** to set up basic configuration.
- **Operation Command Words (OCW1–OCW3)** to control behavior during runtime.

1 Programming the 8259A PIC

The 8259A must be programmed before it can manage interrupts properly. Programming uses two categories of command words:

Two Categories of Commands

- **Initialization Command Words (ICW):** Sent **before** normal operation to define the basic setup.
- **Operation Command Words (OCW):** Sent **during** normal operation to control and modify behavior.

Big Picture: Why Programming is Needed

The 8259A PIC does not work automatically after power-up. Initially, it does not know:

- How many interrupt sources exist
- Whether it is a master or a slave
- Which interrupt numbers it should send to the CPU
- How interrupt priorities should be handled

Therefore, the CPU must **explicitly program** the 8259A so that it knows:

- **How to behave** (initialization phase)
- **How to react** during normal operation

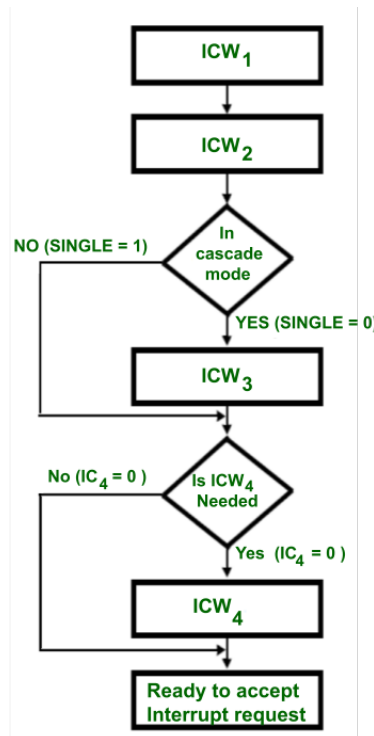


Figure 1: Initialization Command Word Format Overview

2 Initialization Command Words (ICW)

2.1 General Initialization Rules

General Initialization Logic (Easy to Remember)

Initialization is a **step-by-step teaching process** for the PIC.

- All ICWs are sent when $A_0 = 0$.
- The PIC **must always receive ICW1 first**.
- ICW2 **must follow ICW1** because the PIC needs to know interrupt vector numbers.
- ICW3 is needed **only if multiple PICs are used** (cascade mode).
- ICW4 is optional and is used only if ICW1 requests it.

In short:

ICW1 → ICW2 → (ICW3) → (ICW4)

2.2 ICW1: Basic Control Word

ICW1 is the **most important initialization command** because it decides the **overall structure of the interrupt system**. It tells the PIC:

- Whether it works alone or with other PICs
- What kind of interrupt signals it should accept

- Whether additional configuration (ICW4) will follow

Key Fields in ICW1

- **IC4 bit:** If set to 1, the system will use **ICW4**.
- **SNGL bit:**
 - 1 → Single PIC system
 - 0 → Cascade system (**must send ICW3**)
- **LTIM bit:**
 - 0 → **Edge-triggered** interrupt inputs
 - 1 → **Level-triggered** interrupt inputs
- Bits A_7, A_6, A_5 and **ADI:** “Don’t care” for **8086 to Pentium 4** (used with 8085).

How to Think About ICW1

Think of ICW1 as answering the question:

“What kind of interrupt system am I building?”

Once ICW1 is set, the PIC expects the remaining ICWs accordingly.

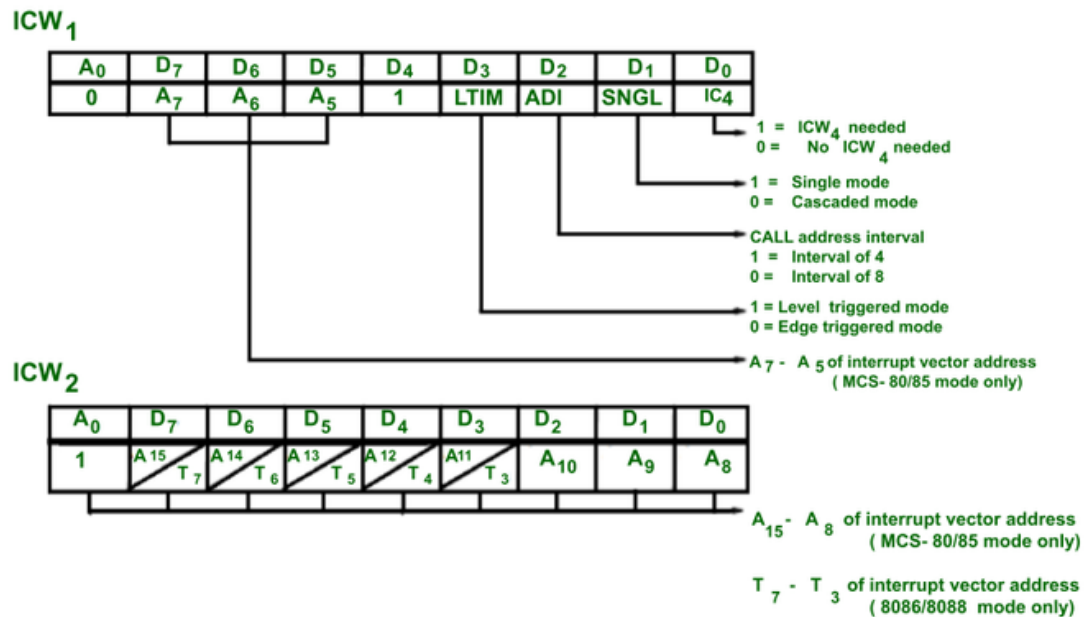


Figure 2: ICW1 and ICW2 Format and Bit Definitions

2.3 ICW2: Interrupt Vector (Type) Number

What ICW2 Does

- ICW2 defines the **base interrupt vector number**.
- Each interrupt line **IR0–IR7** maps to a vector using this base.

Why ICW2 Matters

When an interrupt occurs, the CPU jumps to an **interrupt service routine (ISR)** based on an interrupt number.

ICW2 defines the **starting interrupt number**. The PIC automatically adds the interrupt line number (IR0–IR7) to this base.

Example (Conceptual): If ICW2 sets a base interrupt number, then:

- IR0 maps to the first ISR
- IR1 maps to the next ISR
- IR7 maps to the eighth ISR

2.4 ICW3: Cascade Configuration (Only in Cascade Mode)

ICW3 is required only when the system uses **more than one PIC**. In such systems:

- One PIC acts as the **master**
- Other PICs act as **slaves**

The master receives the interrupt first, but the actual request may come from a slave.

Meaning of ICW3

- **Master PIC:** ICW3 indicates **which IR line(s)** have a slave attached.
- **Slave PIC:** ICW3 stores the **slave ID** (which tells the master where it is connected).

Simple Analogy

Think of the master PIC as a **manager** and slave PICs as **assistants**. ICW3 tells the manager:

- Which assistant is connected
- On which line the assistant reports

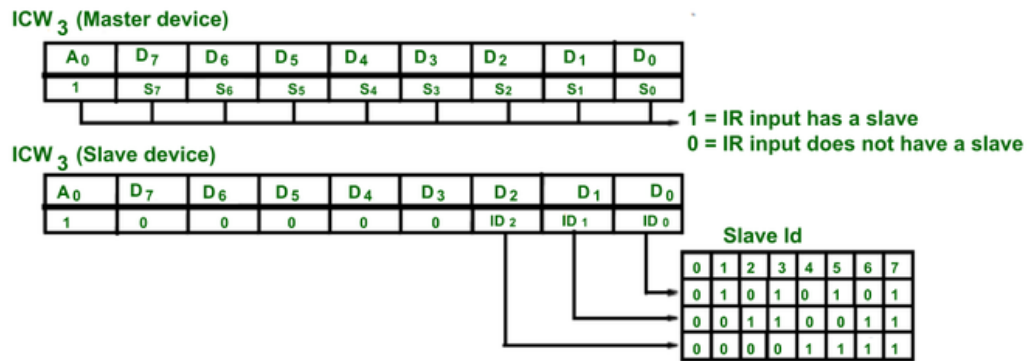


Figure 3: ICW3 Format for Master and Slave PICs

2.5 ICW4: Advanced Options (8086–Pentium 4)

ICW4 is programmed for 8086–Pentium 4 systems; its **rightmost bit must be 1** to select operation.

Important Options in ICW4

- **SFNM (Special Fully Nested Mode):**
Allows a high-priority interrupt from a slave to be recognized by the master even while the master is processing another interrupt from a slave.
- **BUF and M/S:**
Select **buffered or non-buffered** operation and configure **master/slave** in buffered mode.
- **AEIO (Automatic EOI):**
 - Automatically clears the in-service status after interrupt handling.
 - Does **not** change interrupt priority order.

Why ICW4 is Optional

ICW4 is mainly used to:

- Simplify interrupt handling (AEIO)
- Improve performance in complex cascade systems (SFNM)

If ICW4 is not required, the system can still function correctly.

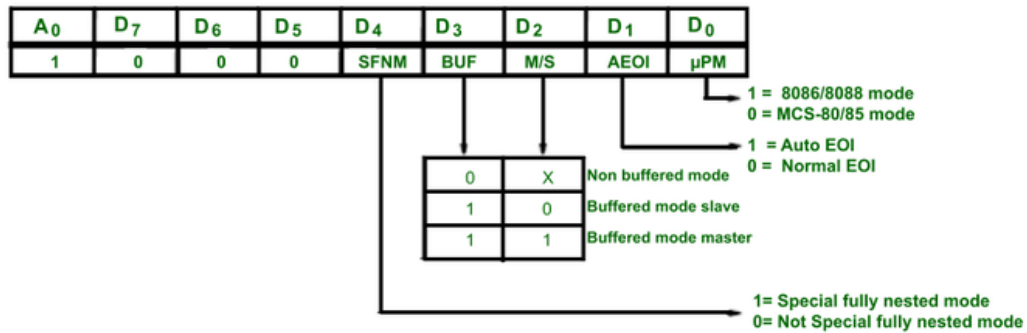


Figure 4: ICW4 Format and Configuration Options

3 Operation Command Words (OCW)

What Happens After Initialization?

Once initialization is complete, the PIC enters **normal operation**. During this phase:

- Interrupts occur dynamically
- Devices request service unpredictably

OCWs allow the CPU to **control and adjust** the PIC while the system is running.

Operation command words control the PIC **during normal operation**.

Address Line Selection (Very Common Question)

- OCW1 is selected when $A_0 = 1$.
- OCW2 and OCW3 are selected when $A_0 = 0$.

3.1 OCW1: Interrupt Mask Register (IMR)

OCW1 is used to **set/read the interrupt mask register**.

OCW1 (IMR) Behavior

- Each mask bit corresponds to an interrupt line **IR0–IR7**.
- Mask bit = 1 → that interrupt is **disabled (turned off)**.
- Mask bit = 0 → that interrupt is **enabled**.
- Reading OCW1 gives the **current IMR state**.

Real-Life Meaning of Masking

Masking an interrupt means **temporarily ignoring a device**.

This is useful when:

- A device is faulty
- A critical section of code is executing
- Interrupt priority needs adjustment

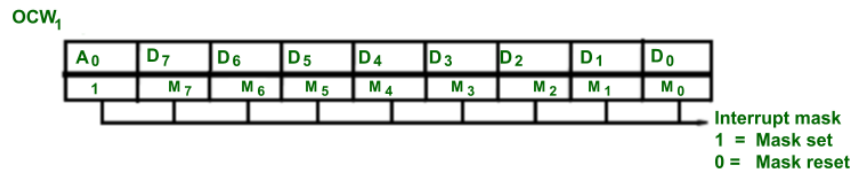


Figure 5: OCW1 - Interrupt Mask Register Format

3.2 OCW2: End of Interrupt (EOI) and Priority Control

OCW2 is mainly used when **AEOI was not selected** in ICW4. It controls how the PIC finishes an interrupt and manages priority.

An interrupt is considered **active** until the PIC is told that it has been serviced. If this signal is not given:

- New interrupts may be blocked
- Priority logic may fail

OCW2 provides this signal using **End-of-Interrupt (EOI)** commands.

Nonspecific EOI

Nonspecific End-of-Interrupt

- Command sent by the ISR (Interrupt Service Program) to signal **EOI**.
- PIC automatically figures out which interrupt level was active and clears it.
- Result: that interrupt can happen again, and lower-priority interrupts can proceed.

Specific EOI

Specific End-of-Interrupt

- Programmer explicitly selects which interrupt line to clear.
- Selection is done using bits $L_2 - L_0$.

Rotating Priority Variants

Rotation Modes (OCW2)

- **Rotate-on-Nonspecific EOI:** Same as nonspecific EOI, but after clearing, the cleared level becomes the **lowest priority**.
- **Rotate-on-Specific EOI:** Same as specific EOI, but also enables rotating priority.
- **Rotate-on-Automatic EOI:** Automatic EOI with rotating priority; a **CLEAR** command disables this mode.
- **Set Priority:** Programmer sets the **lowest priority** interrupt input using $L_2 - L_0$.

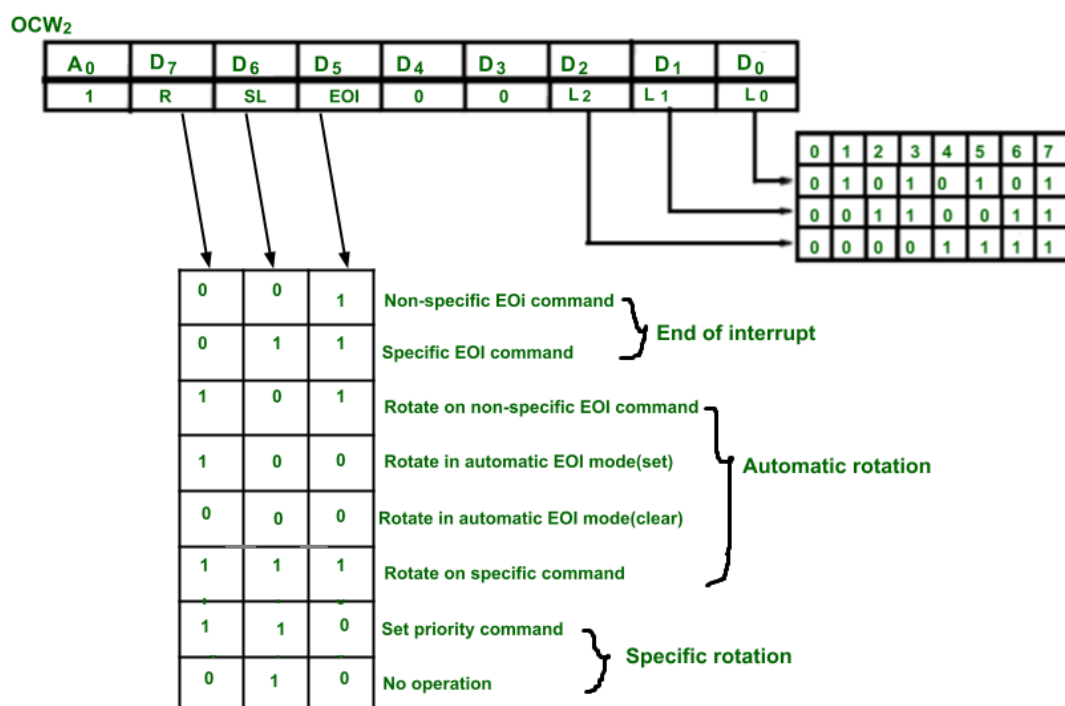


Figure 6: OCW2 Format and EOI Command Options

3.3 OCW3: Poll Mode and Register Selection

OCW3 selects which internal register is read and controls poll mode.

What OCW3 Controls

- Selects which internal register is accessed when the CPU reads the PIC.
- Enables or disables **poll mode**.

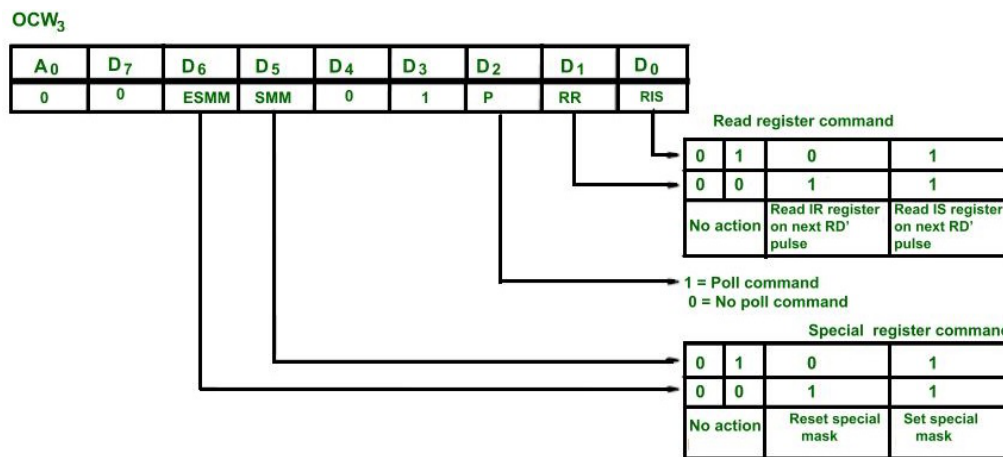


Figure 7: OCW3 Format and Poll Word Structure

Poll Mode Concept

Poll Mode (Idea)

Instead of using the **INT line**, the CPU asks the PIC directly (via software) whether any interrupt request is pending. If $P = 1$, the CPU performs a read from 8259A and receives a poll word.

Why Poll Mode Exists

Polling is useful when:

- Hardware interrupts are disabled
- The system is simple or under debugging
- The CPU wants full control over interrupt checking

Poll Word Format

Poll Word Bit Meaning

- **Bit 7:**
 - 1 → valid interrupt request pending
 - 0 → no pending interrupt
- **Bits 6–3:** Unused
- **Bits 2–0:** Which IR input (IR0–IR7) has the **highest priority active request**

Mini Revision Sheet (1-minute)

Fast Recall

- **ICW** = setup (before operation), **OCW** = control (during operation)
- Power-up must send: **ICW1 + ICW2**
- Cascade? → send **ICW3**
- Need ICW4? → ICW1 bit **IC4=1**
- **OCW1** ($A_0 = 1$): Mask interrupts (IMR)
- **OCW2** ($A_0 = 0$): EOI + rotation + priority
- **OCW3** ($A_0 = 0$): Poll mode + register read selection